Study Guide

Data Balancing and Augmentation Strategies
for Lithography Hotspot Detection

A beginner's walkthrough of the AI/ML concepts and the project itself, from zero prior knowledge

Written for: Group 11 (Priyanuj Boruah, Kartik Aggarwal)
BEVD402L – AI and Machine Learning for IC

How to use this guide

This is not the assignment report – it is a plain-language explainer, written assuming you know *nothing* about AI/ML going in. Read it top to bottom once, then use it as a reference when someone asks "wait, what does that actually mean?" about any part of the project. Every technical term is introduced with a real-world analogy before it's used seriously.

Contents

1 Part 1: AI and Machine Learning, From Zero **3**
1.1 What is "Machine Learning" really? . . . 3
1.2 The two phases: Training and Testing . . . 3
1.3 What is a Neural Network? . . . 3
1.4 What is a CNN (Convolutional Neural Network)? . . . 4
1.5 Binary Classification . . . 4
1.6 Precision, Recall, and Why "Accuracy" Can Lie To You . . . 4

2 Part 2: The Real-World Problem – Lithography Hotspots **5**
2.1 How is a computer chip made? . . . 5
2.2 What is a "hotspot"? . . . 6
2.3 The ICCAD-12 Dataset . . . 6

3 Part 3: Our Specific Problem – Class Imbalance **6**
3.1 What does "imbalanced" mean here? . . . 6
3.2 Why does this matter for training, specifically? . . . 7

4 Part 4: The Six Strategies We Compared **7**

5 Part 5: What We Actually Built **9**

6 Part 6: What We Found – Explained Simply **9**
6.1 Headline result . . . 9
6.2 The most dramatic finding: Benchmark 5 . . . 9
6.3 A genuinely surprising finding: augmentation can backfire . . . 10
6.4 What the confusion matrix numbers mean, concretely . . . 10

7 Glossary (Quick Reference) **10**

1 Part 1: AI and Machine Learning, From Zero

1.1 What is "Machine Learning" really?

Ordinary computer programs work from rules a human writes: *"if the temperature is above 30, turn on the fan."* Machine learning (ML) flips this around. Instead of a human writing the rule, you show the computer thousands of examples, and it *figures out the rule itself.*

Analogy

Imagine teaching a child to tell cats from dogs. You don't hand them a rulebook ("if it has pointy ears and whiskers and is under 40cm long..."). You just show them hundreds of photos, say "cat" or "dog" each time, and eventually they start guessing correctly on photos they've never seen. That's exactly what machine learning does – except the "child" is a mathematical model, and the "photos" are training data.

1.2 The two phases: Training and Testing

Every ML project has two distinct phases, and mixing them up is the single most common beginner mistake:

- **Training** – the model looks at examples *with the correct answer attached* (called *labels*) and adjusts itself to get better at predicting that answer.
- **Testing (or evaluation)** – the model is shown *new* examples it has never seen before, *without* the answer, and we check how many it gets right.

Why the split matters

If you test the model on the same examples it trained on, it can simply "memorize" the answers instead of actually learning the underlying pattern – like a student who memorizes yesterday's exam questions verbatim instead of understanding the topic. That's called **overfitting**, and it's why every serious ML project keeps training data and test data strictly separate. In this project, this distinction is critical: we only ever apply our balancing/augmentation tricks to the *training* data; the test data is always left exactly as it was found.

1.3 What is a Neural Network?

A neural network is one particular kind of ML model, loosely inspired by how brain cells (neurons) connect to each other. Don't worry about the biology – think of it as a big mathematical function made of layers of simple building blocks, where:

- Each "neuron" takes some numbers in, multiplies them by learned weights, adds them up, and passes the result through a simple non-linear switch (called an **activation function**).
- Layers of neurons are stacked, so the output of one layer feeds into the next.
- **Training** means slowly adjusting all those weights (there can be thousands to billions of them) so that the network's final output matches the correct label as often as possible.

Analogy

Think of a factory assembly line with many stations. Raw material (the image) enters at one end. Each station does a small transformation – highlighting edges, detecting shapes, combining patterns – and passes its output to the next station. By the final station, the "product" is a single decision: *hotspot* or *not a hotspot*. Training is the process of tuning every station on the line so the final product is usually correct.

1.4 What is a CNN (Convolutional Neural Network)?

A CNN is a neural network specialized for images. The key building block is a **convolution**: a small sliding window (e.g. 3 pixels by 3 pixels) that scans across the image looking for a specific small pattern – an edge, a corner, a line of a certain thickness. Many such windows (called **filters**) run in parallel, each learning to detect a different small pattern.

- **Convolution layer:** slides small filters over the image to detect local patterns (edges, corners, textures).
- **Pooling layer:** shrinks the image down (e.g. takes the strongest signal in every 2×2 block), which both reduces computation and makes the network care less about exactly *where* a pattern appeared.
- **Fully-connected layer:** at the very end, takes all the detected patterns and combines them into a final decision.

Stacking several convolution+pooling stages lets early layers detect simple things (edges) and later layers combine those into complex things (a "pinched wire" shape that indicates a hotspot).

Why CNNs, and not something simpler, for this project

A layout image is just a grid of pixels. A CNN is good at this task because a hotspot is defined by a *local geometric pattern* (two lines getting too close together, a corner that's too sharp) – exactly the kind of small, spatial pattern that convolution filters are built to detect, wherever it appears in the image.

1.5 Binary Classification

Our task has exactly two possible answers: *Hotspot (HS)* or *Non-Hotspot (NHS)*. This is called **binary classification** ("binary" = two options). The network's very last layer outputs a single number between 0 and 1 (think of it as "how confident am I that this is a hotspot, from 0% to 100%"), and we typically call it a hotspot if that number is above 0.5.

1.6 Precision, Recall, and Why "Accuracy" Can Lie To You

This is the single most important idea for understanding this entire project, so we'll spend real time on it.

Accuracy is the simplest possible score: what fraction of all predictions were correct? It sounds reasonable, but it has a fatal flaw when one class is much rarer than the other – which is exactly our situation (see Part 3).

Analogy

Imagine a medical test for a rare disease that only 1 in 1,000 people actually have. A "test" that always says "you're healthy," no matter what, will be correct 999 times out of 1,000 – 99.9% accuracy! But it is completely useless: it will never, ever catch the one person who is actually sick. High accuracy, zero usefulness.

This is precisely the trap in lithography hotspot detection: real hotspots are rare (Part 3), so a model that just says "not a hotspot" every single time can score above 99% raw accuracy on some of our benchmarks while catching *zero* real hotspots. That is why this whole project uses different scores instead:

- **Recall (a.k.a. Sensitivity):** of all the *actual* hotspots, what fraction did we correctly catch? This is the "did we miss the disease" score.
- **Specificity:** of all the actual *non*-hotspots, what fraction did we correctly leave alone? This is the "did we cry wolf" score.
- **Precision:** of everything we *flagged* as a hotspot, what fraction actually were? High precision means few false alarms.
- **Balanced Accuracy:** the average of Recall and Specificity. This is the project's main scorecard, precisely because it can't be gamed by just always predicting the majority class – to score well, a model must do well on *both* classes.
- **F1-score:** a combined score balancing Precision and Recall, useful when you care about both false alarms and missed detections together.

The Confusion Matrix

All of the above scores come from just four numbers, usually laid out in a 2×2 grid called a **confusion matrix**:

		Model Predicted	
		Hotspot	Non-Hotspot
Actually Was	Hotspot	True Positive (TP)	False Negative (FN)
	Non-Hotspot	False Positive (FP)	True Negative (TN)

FN (a real hotspot we missed) is usually the most dangerous error in this domain – it slips through to manufacturing. FP (a false alarm) just costs some wasted verification time.

2 Part 2: The Real-World Problem – Lithography Hotspots

2.1 How is a computer chip made?

A modern chip has billions of microscopic wires and transistors packed onto a few square centimeters of silicon. The pattern of these wires is "printed" onto the silicon using a process called **photolithography** – shining light through a stencil (called a **mask**) coated with the desired circuit pattern, similar in spirit to how a photographic negative projects an image, but at a scale of billionths of a metre.

2.2 What is a "hotspot"?

As chip features get smaller and smaller, they start to approach the same size as the *wavelength of light* being used to print them. When a wire (or the gap between two wires) is on the edge of what the light can reliably resolve, small pattern variations can cause a real manufacturing defect:

- **Pinching (open circuit):** two ends of a wire that were supposed to connect end up separated – like a bridge that doesn't quite meet in the middle.
- **Bridging (short circuit):** two separate wires that were supposed to stay apart end up merging together – like two lanes of a highway accidentally merging.

A **hotspot** is a small region of the chip layout that is at high risk of one of these failures once actually printed. Figure 1 of the reference paper shows real examples: a design's intended pattern ("Original") next to what actually got printed ("Printed"), with the defect circled.

Why we bother detecting these early

Finding a hotspot *after* manufacturing means the chip is already defective – expensive, wasted silicon. Finding it during *design*, before sending the layout to the fab, means engineers can just tweak that one small region and re-check, which is far cheaper. The gold-standard way to check is full optical simulation of how light will behave – but that is slow and computationally very expensive to run across an entire chip. This project (like the wider field it belongs to) is about using a fast, approximate, ML-based screening step to flag *likely* hotspots quickly, so full simulation only needs to double-check the suspicious regions.

2.3 The ICCAD-12 Dataset

To let researchers compare their methods fairly, a standard benchmark dataset was released in 2012, called **ICCAD-12**. It contains five separate collections ("Benchmark 1" through "Benchmark 5"), each a set of small image clips cut from real chip layouts, where every clip is labeled either Hotspot (HS) or Non-Hotspot (NHS). This is the exact dataset our whole project uses, and it is what makes results from different student groups (and different research papers) directly comparable.

3 Part 3: Our Specific Problem – Class Imbalance

3.1 What does "imbalanced" mean here?

In the real world, the overwhelming majority of any chip design is completely fine – hotspots are, by definition, rare exceptions. This shows up directly in the dataset:

Benchmark	Hotspot images	Non-Hotspot images	Ratio
B1	226	4,679	1 hotspot per 21 images
B2	498	41,298	1 hotspot per 83 images
B3	1,808	46,333	1 hotspot per 26 images
B4	177	31,890	1 hotspot per 180 images
B5	41	19,327	1 hotspot per **471** images

On Benchmark 5, for every single real hotspot, there are 471 non-hotspot images. This is what we mean by **severe class imbalance**, and it is the central obstacle this whole project is about.

Analogy

Imagine trying to teach someone to recognize a rare bird species by showing them 471 pictures of pigeons for every 1 picture of the rare bird. If they just learned to say "pigeon" every single time, they'd be right 99.8% of the time – and would never learn to recognize the rare bird at all. A neural network trained naively on this kind of data has exactly the same temptation: predicting the majority class every time is a very easy way to rack up high raw accuracy.

3.2 Why does this matter for training, specifically?

Neural networks learn by seeing many examples and gradually reducing their mistakes. If 99.8% of what they see is one class, the easiest way to reduce mistakes is to just lean heavily towards predicting that class – the rare class barely gets enough "votes" during training to matter. Our project's whole job is to fix this imbalance in the *training data* so the model actually has to learn what a real hotspot looks like.

4 Part 4: The Six Strategies We Compared

This is Group 11's specific research angle: *comparing different ways of fixing the training data's imbalance*, to see which works best. Here is each strategy explained with a plain-language analogy.

0. Baseline – Do Nothing

Train on the data exactly as it comes, imbalance and all. This is our reference point – every other strategy is judged by how much (or how little) it improves on this.

1. Random Undersampling (RUS)

Idea: Throw away most of the majority (non-hotspot) examples until the two classes are more equal in number.

Analogy

If you have 4,700 pigeon photos and 226 rare-bird photos, just randomly delete pigeon photos until you have roughly 226 of each. Now the training set is balanced – but you've thrown away a lot of pigeon photos you could have learned from, and you haven't added a single new rare-bird photo.

Risk: If the rare class had very few examples to begin with, you end up with a tiny total training set, and the model may not have enough information to learn from at all.

2. Random Oversampling (ROS)

Idea: Instead of deleting majority examples, duplicate minority (hotspot) examples until the classes are more equal.

Analogy

Instead of deleting pigeon photos, make 20 identical copies of each rare-bird photo you have, so the count matches. You keep everything, but the rare-bird photos are now exact repeats.

Risk: The model might start "memorizing" those exact duplicated images rather than learning the general pattern – overfitting to a small handful of examples.

3. Oversampling + Augmentation

Idea: Same as ROS, but instead of making *identical* copies, apply small, label-preserving changes to each duplicate – flipping it left-right, flipping it top-bottom, or rotating it 90/180/270 degrees.

Analogy

Instead of 20 identical photocopies of the same rare bird photo, you make 20 copies but mirror some, flip some upside down, and rotate some. It's still recognizably the same bird, but the model sees more *variety*, making it a little harder to just memorize pixels.

Why flips/rotations specifically? A hotspot is still a hotspot if you flip or rotate the whole image – it's the same physical pattern, just viewed from a different orientation, so this doesn't create fake/incorrect examples.

4. Balanced Mini-Batch Sampling

Idea: Don't touch the dataset at all. Instead, change how training examples are *picked* in each small batch during training, so that every batch shown to the model has roughly equal hotspot and non-hotspot examples.

Analogy

Instead of editing your photo collection, you change how you *study* it: every study session, you deliberately pull an equal number of pigeon and rare-bird photos from your (untouched, full) collection. No photo was deleted or copied – you just changed your study habits.

Why this often works well: it gets the benefit of balanced exposure during training without physically discarding data (like RUS) or risking memorized duplicates (like ROS).

5. Combined Strategy

Idea: Do a bit of both – moderately reduce the majority class *and* moderately increase the (augmented) minority class, meeting somewhere in the middle, rather than going to either extreme.

5 Part 5: What We Actually Built

1. **Got the real data.** Downloaded the actual ICCAD-12 dataset (all five benchmarks, tens of thousands of real layout images) and confirmed the hotspot/non-hotspot counts matched the published reference paper.
2. **Built one shared "brain".** Implemented a single lightweight CNN (about 8,600 tunable numbers – tiny by modern AI standards, deliberately so it trains fast) that every strategy uses identically. This is important: if every strategy uses the exact same network, any performance difference we see must come from the *training data* itself, not from a better or worse network design. That is what makes this a fair, controlled experiment.
3. **Implemented all six strategies** as code, each one taking the raw imbalanced training images in and handing back a (possibly resampled/augmented) training set, never touching the test images.
4. **Trained and tested 105 times.** 5 benchmarks $\times$ 6 strategies = 30 main experiments, plus 75 more experiments where we varied *how strongly* to rebalance the data (to see if going halfway is better than going all the way).
5. **Measured everything that matters.** For every one of those 105 runs, we recorded the full confusion matrix, balanced accuracy, precision, recall, specificity, and F1-score – never just raw accuracy, for the reasons explained in Part 1.
6. **Turned the numbers into pictures and a paper.** Generated charts and tables, and wrote them up as a proper IEEE-style research report.

6 Part 6: What We Found – Explained Simply

6.1 Headline result

Averaged across all five benchmarks:

Strategy	Average Balanced Accuracy
Balanced Mini-Batch Sampling	**0.945 (best)**
Combined	0.926
Random Oversampling	0.923
Oversampling + Augmentation	0.887
Baseline (do nothing)	0.844
Random Undersampling	**0.779 (worst)**

In plain terms: **just re-sampling which examples get shown to the model in each training batch (Strategy #4) worked better, on average, than any other trick** – including fancier ones that alter the dataset itself. And somewhat surprisingly, **the strategy that most directly "fixes" the imbalance by deleting data (undersampling) was actually the single worst performer.**

6.2 The most dramatic finding: Benchmark 5

Benchmark 5 is the extreme case – only 26 real hotspot training images, against 2,716 non-hotspot ones. Here's what happened:

- **Random Undersampling:** caught **0** of 41 real hotspots in the test set. It predicted "not a hotspot" every single time, no matter what we fed it – even though we had artificially balanced its training set to 50/50! We even tried every possible balance level (from mild to full) and it stayed stuck at 0 hotspots caught, every time.
- **Balanced Mini-Batch Sampling:** caught **40 of 41** real hotspots (97.6% recall).

Takeaway

Why did undersampling fail so badly? Because it only ever had 26 *original* hotspot images to work with – deleting non-hotspot images doesn't create a single new piece of information about what a hotspot looks like. If you only ever show a child 26 photos of a rare bird, no amount of throwing away pigeon photos will suddenly teach them what that bird looks like. The lesson: **when data is critically scarce, the fix has to add information (or at least preserve all of it), not just remove the competing class.**

6.3 A genuinely surprising finding: augmentation can backfire

We expected "oversampling + augmentation" (Strategy #3) to always beat plain oversampling (Strategy #2), since flipping/rotating adds variety. That was true on Benchmarks 2 and 3 (augmentation clearly helped). But on Benchmark 5 – the one with only 26 original hotspot images – augmentation actually made things *worse* than plain duplication (balanced accuracy 0.707 vs. 0.914).

Takeaway

When you have very few original examples, almost every hotspot image the network sees during training ends up being a flipped/rotated copy of one of those few originals. The model can end up learning quirks of the flipping/rotating process itself, rather than the real geometric pattern of a hotspot – and that hurts it on the real (unflipped) test images. **The lesson: a technique that helps in one setting can hurt in another – there's no single "best" strategy that works everywhere, which is exactly the nuanced answer our research question was looking for.**

6.4 What the confusion matrix numbers mean, concretely

On Benchmark 2, the do-nothing baseline caught only 197 of 498 real hotspots (missed 301!) while being extremely cautious about false alarms. Switching to the Combined strategy caught 495 of 498 real hotspots, at the cost of a modest number of extra false alarms (415 non-hotspots incorrectly flagged, out of 41,713 total). In a real manufacturing context, missing 301 real defects is a far more expensive mistake than double-checking 415 patterns that turned out to be fine – which is exactly the kind of trade-off Balanced Accuracy (and the confusion matrix) is designed to make visible.

7 Glossary (Quick Reference)

Accuracy
Fraction of all predictions that were correct. Misleading when classes are imbalanced.

Balanced Accuracy
Average of Recall and Specificity; the fair scorecard used throughout this project.

Baseline
Training with no balancing intervention – the reference point.

Class imbalance
When one label (here, non-hotspot) vastly outnumbers the other (hotspot) in the data.

CNN (Convolutional Neural Network)
A neural network specialized for images, using sliding filters to detect local visual patterns.

Confusion matrix
The 2×2 table of TP/FP/FN/TN counts that every other metric is computed from.

Data augmentation
Creating variations of existing examples (e.g. flips, rotations) to add diversity without collecting new data.

F1-score
Combined score balancing Precision and Recall.

Hotspot (HS)
A layout region at high risk of a real manufacturing defect once printed.

Non-Hotspot (NHS)
A layout region that is expected to print correctly.

Oversampling
Duplicating minority-class examples to reduce imbalance.

Overfitting
When a model memorizes training examples instead of learning general patterns, and performs worse on new data.

Precision
Of everything flagged positive, what fraction was correct.

Recall (Sensitivity)
Of all real positives, what fraction was correctly caught.

Specificity
Of all real negatives, what fraction was correctly left alone.

Test set
Data held back and never used for training, used only to check final performance.

Training set
Data (with correct labels) shown to the model so it can learn.

Undersampling
Discarding majority-class examples to reduce imbalance.